

Лабораторная работа №1

Индексы и EXPLAIN ANALYZE в PostgreSQL

Цель работы

В рамках лабораторной работы вы:

- научитесь анализировать выполнение SQL-запросов с помощью EXPLAIN и EXPLAIN ANALYZE;
 - познакомитесь с основными способами доступа к данным в PostgreSQL;
 - поймёте, когда и зачем используются индексы;
 - научитесь выбирать индекс под конкретный запрос;
 - разберётесь с составными, частичными и индексами по выражению;
 - увидите, почему наличие индекса не гарантирует его использование PostgreSQL;
 - примените полученные знания для анализа и оптимизации собственного сервиса.
-

Главное правило лабораторной работы

Нельзя считать запрос оптимизированным только потому, что был создан индекс.

Любое изменение должно быть подтверждено измерением:

EXPLAIN ANALYZE до
↓
изменение
↓
EXPLAIN ANALYZE после
↓
сравнение результата

На протяжении всей работы необходимо не просто создавать индексы, а объяснять:

1. Почему PostgreSQL выбрал текущий план выполнения?
 2. Почему выбранный индекс подходит или не подходит для запроса?
 3. Как изменился план после оптимизации?
 4. Как изменилось время выполнения запроса?
-

Теоретический минимум

Перед выполнением лабораторной работы рекомендуется ознакомиться со следующими понятиями:

- EXPLAIN;
- EXPLAIN ANALYZE;
- Sequential Scan;
- Index Scan;
- Bitmap Index Scan;
- Bitmap Heap Scan;
- Index Only Scan;
- B-tree index;
- Composite index;
- Partial index;
- Expression index;
- селективность запроса;
- стоимость операций чтения и записи.

Часть 1. Подготовка тестовой базы данных

В первой части лабораторной работы используется общая тестовая база данных.

Цель этой части — познакомиться с различными планами выполнения запросов в контролируемой среде.

Задание 1. Создание таблицы

Создайте таблицу orders.

```
CREATE TABLE orders (  
  id BIGSERIAL PRIMARY KEY,  
  
  user_id BIGINT NOT NULL,  
  
  product_id BIGINT NOT NULL,  
  
  status VARCHAR(20) NOT NULL,  
  
  amount NUMERIC(10, 2) NOT NULL,  
  
  created_at TIMESTAMP NOT NULL,
```

```
        updated_at TIMESTAMP NOT NULL
    );
```

Задание 2. Генерация данных

Заполните таблицу тестовыми данными.

Минимальный рекомендуемый объём данных:

1 000 000 записей

Пример:

```
INSERT INTO orders (
    user_id,
    product_id,
    status,
    amount,
    created_at,
    updated_at
)
SELECT
    (random() * 100000)::BIGINT,
    (random() * 10000)::BIGINT,
    (
        ARRAY[
            'NEW',
            'PAID',
            'DELIVERED',
            'CANCELLED'
        ]
    )[floor(random() * 4 + 1)],
    random() * 10000,
    NOW() - (random() * INTERVAL '2 years'),
    NOW()
FROM generate_series(1, 1000000);
```

После загрузки данных выполните:

```
ANALYZE orders;
```

Часть 2. Знакомство с EXPLAIN

Задание 3. Анализ плана выполнения

Выполните запрос:

```
EXPLAIN
SELECT *
FROM orders
WHERE user_id = 123;
```

Ответьте на вопросы:

1. Какой план выполнения выбрал PostgreSQL?
 2. Какой тип сканирования используется?
 3. Как PostgreSQL планирует найти необходимые строки?
-

Задание 4. EXPLAIN ANALYZE

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123;
```

Изучите результат выполнения запроса.

Найдите в плане:

- тип сканирования;
- Planning Time;
- Execution Time;
- количество фактически возвращённых строк (actual rows);
- количество строк, отброшенных фильтром (Rows Removed by Filter), если присутствует.

Ответьте на вопросы:

1. Чем EXPLAIN отличается от EXPLAIN ANALYZE?
 2. Почему EXPLAIN ANALYZE может выполняться значительно дольше, чем EXPLAIN?
 3. Чем отличаются предполагаемые (estimated) и фактические (actual) значения количества строк?
-

Часть 3. Sequential Scan

Задание 5. Последовательное сканирование

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders;
```

Затем:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE amount > 0;
```

Ответьте на вопросы:

1. Какой тип сканирования используется?
2. Почему PostgreSQL выбирает Sequential Scan?
3. Всегда ли Sequential Scan является плохим решением?

Сделайте вывод:

В каких случаях последовательное сканирование таблицы может быть эффективнее использования индекса?

Часть 4. Первый индекс

Задание 6. B-tree индекс

Создайте индекс:

```
CREATE INDEX idx_orders_user_id
ON orders(user_id);
```

После создания индекса выполните:

```
ANALYZE orders;
```

Повторите запрос:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123;
```

Сравните результаты до и после создания индекса.

Заполните таблицу:

Метрика	До индекса	После индекса
Тип Scan		
Execution Time		
Количество обработанных строк		
Используемый индекс		

Ответьте на вопросы:

1. Изменился ли план выполнения?
 2. Использует ли PostgreSQL созданный индекс?
 3. Насколько изменилось время выполнения запроса?
-

Часть 5. Индекс используется не всегда

Задание 7. Индекс и низкая селективность

Создайте индекс:

```
CREATE INDEX idx_orders_status  
ON orders(status);
```

Выполните:

```
EXPLAIN ANALYZE  
SELECT *  
FROM orders  
WHERE status = 'PAID';
```

Повторите запрос для других значений status.

Например:

```
WHERE status = 'NEW';  
WHERE status = 'DELIVERED';  
WHERE status = 'CANCELLED';
```

Ответьте на вопросы:

1. Во всех ли случаях PostgreSQL использует индекс?
 2. Почему PostgreSQL может выбрать Sequential Scan, несмотря на наличие индекса?
 3. Как количество возвращаемых строк влияет на выбор плана выполнения?
-

Часть 6. Селективность

Задание 8. Исследование распределения данных

Определите количество записей для каждого значения status.

```
SELECT  
    status,  
    COUNT(*)  
FROM orders  
GROUP BY status;
```

Для каждого значения выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE status = '...';
```

Проанализируйте:

- сколько строк возвращает запрос;
- какой процент таблицы составляет результат;
- используется ли индекс;
- какой тип сканирования выбирает PostgreSQL.

Сделайте вывод:

Как селективность условия влияет на эффективность индекса?

Часть 7. Range Query

Задание 9. Индекс для диапазонного запроса

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE created_at > NOW() - INTERVAL '7 days';
```

Создайте индекс:

```
CREATE INDEX idx_orders_created_at
ON orders(created_at);
```

Повторите запрос.

Затем исследуйте разные временные диапазоны:

```
WHERE created_at > NOW() - INTERVAL '1 day';
WHERE created_at > NOW() - INTERVAL '1 month';
WHERE created_at > NOW() - INTERVAL '1 year';
```

Ответьте на вопросы:

1. Во всех ли случаях PostgreSQL использует индекс?
 2. Как размер временного диапазона влияет на план выполнения?
 3. В какой ситуации использование индекса становится менее выгодным?
-

Часть 8. Bitmap Scan

Задание 10. Bitmap Index Scan

Подберите запрос, который возвращает среднее количество строк.

Например:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE status = 'NEW';
```

или:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE amount BETWEEN 1000 AND 3000;
```

Попробуйте получить план выполнения вида:

```
Bitmap Heap Scan
  -> Bitmap Index Scan
```

Ответьте на вопросы:

1. Что делает Bitmap Index Scan?
2. Что делает Bitmap Heap Scan?
3. Почему PostgreSQL может использовать Bitmap Scan вместо обычного Index Scan?

Часть 9. Несколько условий и несколько индексов

Задание 11. Использование нескольких индексов

Убедитесь, что существуют индексы:

```
CREATE INDEX idx_orders_user_id
ON orders(user_id);
```

```
CREATE INDEX idx_orders_status
ON orders(status);
```

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123
      AND status = 'PAID';
```


Проанализируйте план выполнения.

Ответьте на вопросы:

1. Использует ли PostgreSQL один индекс или несколько?
 2. Есть ли в плане BitmapAnd?
 3. Почему PostgreSQL может объединять результаты нескольких индексов?
-

Часть 10. Composite Index

Задание 12. Составной индекс

Создайте составной индекс:

```
CREATE INDEX idx_orders_user_status
ON orders(user_id, status);
```

Повторите запрос:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123
AND status = 'PAID';
```

Сравните:

- использование двух отдельных индексов;
- использование составного индекса.

Ответьте на вопросы:

1. Какой вариант оказался эффективнее?
 2. Всегда ли составной индекс лучше нескольких отдельных индексов?
 3. От чего зависит выбор между несколькими индексами и одним составным?
-

Часть 11. Порядок колонок в составном индексе

Задание 13. Влияние порядка колонок

Создайте индекс:

```
CREATE INDEX idx_orders_user_created_at
ON orders(user_id, created_at);
```

Исследуйте следующие запросы.

Запрос 1

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123;
```

Запрос 2

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123
    AND created_at > NOW() - INTERVAL '30 days';
```

Запрос 3

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE created_at > NOW() - INTERVAL '30 days';
```

Затем создайте индекс с обратным порядком колонок:

```
CREATE INDEX idx_orders_created_at_user
ON orders(created_at, user_id);
```

Повторите эксперименты.

Ответьте на вопросы:

1. Как порядок колонок влияет на возможность использования индекса?
 2. Почему (user_id, created_at) и (created_at, user_id) — это разные индексы?
 3. Какой индекс лучше подходит для каждого из исследованных запросов?
-

Часть 12. WHERE + ORDER BY

Задание 14. Индекс и сортировка

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123
ORDER BY created_at DESC;
```

Найдите в плане операцию:

Sort

Создайте индекс:

```
CREATE INDEX idx_orders_user_created_at_desc
ON orders(user_id, created_at DESC);
```

Повторите запрос.

Ответьте на вопросы:

1. Исчезла ли операция Sort?
 2. Почему индекс может использоваться для сортировки?
 3. Почему порядок колонок важен в данном случае?
-

Часть 13. Реальный API-запрос

Задание 15. Pagination Query

Представьте endpoint:

```
GET /users/{user_id}/orders
```

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 123
ORDER BY created_at DESC
LIMIT 20;
```

Ваша задача

Подобрать индекс, который максимально эффективно оптимизирует данный запрос.

Необходимо:

1. Сохранить план выполнения до оптимизации.
 2. Создать подходящий индекс.
 3. Повторно выполнить EXPLAIN ANALYZE.
 4. Сравнить планы выполнения.
 5. Сравнить время выполнения.
 6. Объяснить выбранную структуру индекса.
-

Часть 14. Index Only Scan

Задание 16. Получение данных из индекса

Исследуйте запрос:

```
EXPLAIN ANALYZE
SELECT id, user_id
FROM orders
WHERE user_id = 123;
```

Попробуйте получить:

Index Only Scan

Ответьте на вопросы:

1. Чем Index Only Scan отличается от обычного Index Scan?
2. Почему PostgreSQL может не обращаться к основной таблице?
3. Какие условия необходимы для выполнения Index Only Scan?

Дополнительное задание

Исследуйте использование INCLUDE:

```
CREATE INDEX idx_orders_user_id_include
ON orders(user_id)
INCLUDE (id, status, created_at);
```

Часть 15. Partial Index

Задание 17. Частичный индекс

Предположим, что только небольшая часть заказов имеет статус NEW.

Выполните:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE status = 'NEW'
ORDER BY created_at;
```

Создайте частичный индекс:

```
CREATE INDEX idx_orders_new
ON orders(created_at)
WHERE status = 'NEW';
```

Повторите запрос.

Сравните:

- обычный индекс;
- partial index.

Ответьте на вопросы:

1. В чём преимущество partial index?
 2. В каких случаях partial index будет полезен?
 3. Почему нельзя использовать один partial index для всех типов запросов?
-

Часть 16. Expression Index

Задание 18. Индекс по выражению

Создайте таблицу:

```
CREATE TABLE users (  
    id BIGSERIAL PRIMARY KEY,  
    email VARCHAR(255) NOT NULL  
);
```

Создайте индекс:

```
CREATE INDEX idx_users_email  
ON users(email);
```

Выполните:

```
EXPLAIN ANALYZE  
SELECT *  
FROM users  
WHERE LOWER(email) = 'test@example.com';
```

Ответьте на вопрос:

Почему обычный индекс по email может не использоваться?

Создайте индекс по выражению:

```
CREATE INDEX idx_users_lower_email  
ON users(LOWER(email));
```

Повторите запрос.

Сделайте вывод:

Почему выражение в запросе влияет на возможность использования индекса?

Часть 17. Цена индексов

Задание 19. Влияние индексов на INSERT

Создайте тестовую таблицу без дополнительных индексов.

Измерьте время вставки большого количества записей.

Затем добавьте несколько индексов и повторите вставку аналогичного количества данных.

Ответьте на вопросы:

1. Как изменилось время выполнения INSERT?
 2. Почему?
 3. Что происходит с индексами при добавлении новой строки?
-

Часть 18. Анализ существующих индексов

Задание 20. Неиспользуемые индексы

Посмотрите статистику использования индексов:

```
SELECT
    schemaname,
    relname,
    indexrelname,
    idx_scan
FROM pg_stat_user_indexes
ORDER BY idx_scan;
```

Ответьте на вопросы:

1. Какие индексы использовались чаще всего?
 2. Есть ли индексы, которые не использовались?
 3. Можно ли автоматически считать индекс с `idx_scan = 0` бесполезным?
 4. Какие риски существуют при удалении индекса?
-

Часть 19. Финальное задание на тестовой базе

Задание 21. Оптимизация запроса

Вам дан следующий запрос:

```
SELECT
    id,
    amount,
    status,
    created_at
FROM orders
WHERE user_id = $1
    AND status = 'PAID'
    AND created_at >= NOW() - INTERVAL '30 days'
```

```
ORDER BY created_at DESC  
LIMIT 50;
```

Необходимо:

1. Выполнить EXPLAIN ANALYZE.
 2. Проанализировать текущий план выполнения.
 3. Определить потенциальные проблемы.
 4. Предложить индекс или набор индексов.
 5. Объяснить выбор порядка колонок.
 6. Создать индекс.
 7. Повторно выполнить EXPLAIN ANALYZE.
 8. Сравнить результаты.
-

Часть 20. Работа со своим сервисом

После выполнения заданий на тестовой базе необходимо применить полученные знания к собственному проекту.

Задание 22. Выберите Scaling Entity

Выберите таблицу вашего сервиса, которая потенциально будет активно расти.

Примеры:

- orders;
- messages;
- comments;
- events;
- payments;
- notifications;
- posts;
- bookings;
- transactions.

Опишите:

1. Название таблицы.
 2. За что отвечает данная сущность.
 3. Почему именно эта таблица будет расти быстрее остальных.
 4. Какой потенциальный объём данных вы ожидаете через год.
 5. Какие поля чаще всего используются для поиска и фильтрации.
-

Задание 23. Выберите реальные запросы

Найдите минимум **3 реальных SQL-запроса** вашего сервиса.

Желательно выбрать разные типы запросов.

Query 1

Поиск по конкретному идентификатору:

```
WHERE user_id = ?
```

Query 2

Запрос с несколькими условиями:

```
WHERE ...  
AND ...
```

Query 3

Фильтрация + сортировка + ограничение:

```
WHERE ...  
ORDER BY ...  
LIMIT ...
```

Можно использовать SQL-запросы:

- из логов PostgreSQL;
- из логов ORM;
- полученные из приложения;
- вручную составленные на основе реального endpoint.

Задание 24. Базовое измерение

Для каждого выбранного запроса:

1. Выполните EXPLAIN.
2. Выполните EXPLAIN ANALYZE.
3. Сохраните результат.

Заполните таблицу:

Запрос	Тип Scan	Execution Time	Используемые индексы
Query 1			
Query 2			
Query 3			

Задание 25. Анализ текущих индексов

Получите список существующих индексов вашей базы данных.

Для каждого индекса определите:

- на каких колонках он построен;
- какие запросы могут его использовать;
- используется ли он фактически.

Дополнительно изучите статистику:

```
SELECT
    schemaname,
    relname,
    indexrelname,
    idx_scan
FROM pg_stat_user_indexes
ORDER BY idx_scan;
```

Задание 26. Найдите потенциальную проблему

Проанализируйте три выбранных запроса.

Для каждого ответьте:

1. Есть ли потенциальная проблема с производительностью?
 2. Используется ли Sequential Scan?
 3. Насколько селективен запрос?
 4. Есть ли подходящий индекс?
 5. Можно ли улучшить существующий индекс?
 6. Можно ли изменить сам SQL-запрос?
-

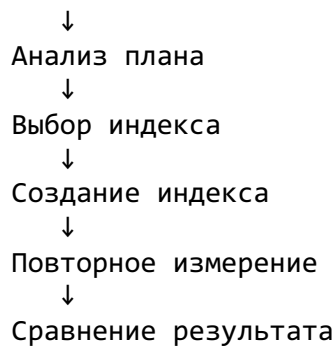
Часть 21. Оптимизация собственного сервиса

Задание 27. Выберите запрос для оптимизации

Выберите минимум **один реальный запрос** вашего сервиса, который потенциально можно улучшить.

Необходимо пройти полный инженерный процесс:

Проблема
↓
Измерение
↓
EXPLAIN ANALYZE



Задание 28. Реализуйте изменение

Добавьте выбранный индекс в проект.

Индекс должен быть добавлен через миграции

Изменение структуры базы данных не должно существовать только в вашей локальной базе данных.

Изменение должно быть воспроизводимым при запуске проекта.

Задание 29. Повторное измерение

После добавления индекса повторите:

EXPLAIN ANALYZE

Сравните результаты.

Метрика	До	После
Тип Scan		
Execution Time		
Actual Rows		
Используемый индекс		

Часть 23. Финальный вопрос

Задание 30. Почему нельзя индексировать всё?

Ответьте на вопрос:

Почему нельзя просто создать индексы на всех колонках таблицы?

В ответе необходимо рассмотреть:

- размер индексов;
 - стоимость хранения;
 - INSERT;
 - UPDATE;
 - DELETE;
 - необходимость обновления индексов;
 - влияние на скорость записи;
 - реальное использование запросами.
-

Что необходимо показать для оценки

1. Репозиторий проекта

В репозитории должны быть:

- миграции с созданными индексами;
 - изменения структуры базы данных;
 - обновлённая документация.
-

2. Отчёт

Необходимо добавить файл:

`docs/lab-01-indexes.md`

Пример структуры отчёта (можно в свободной форме, но информативно)

```
# Лабораторная работа №1
# Индексы и EXPLAIN ANALYZE

## Часть 1. Работа с тестовой базой

### Задание 3. EXPLAIN
...

### Задание 4. EXPLAIN ANALYZE
...
```

Задание 6. Первый индекс

...

Часть 2. Работа со своим сервисом

Scaling Entity

...

Почему была выбрана эта сущность?

...

Исследуемые запросы

Query 1

...